CAPSTONE PROJECT BRIEF · PROJECT 04

Predictive-Maintenance Operations Copilot

Client: Ironside Manufacturing · an industrial equipment operator · a fictional persona for this engagement

Take a real run-to-failure sensor dataset and build an agentic operations copilot for a fleet of machines. It watches every asset, catches degradation early, explains what is wrong in plain language, and drafts the work order and the schedule. Catches the failures that matter. Runs at fleet scale. Inside a budget.

ENGAGEMENT
A client-style build, run end to end.

TIMELINE
4 to 5 one-week sprints, with reviews and retros.

TEAM POSTURE
A small engineering squad sharing one codebase.

DATA
One real public dataset, augmented with synthetic data.

DELIVERABLE
A deployed system, full reports, a 20-minute video, a 1:1 viva.

ASSESSMENT
Five dimensions, with an individual viva gate.

HOW THIS BRIEF WORKS

Read the journey first

This is a client engagement, not a coursework prompt. You run it as a sprint-based delivery and move through eight stages of a product development lifecycle. Read the journey below so the demands in each stage make sense in sequence. Every stage states what you must produce and what a finished version looks like. Nothing here is optional unless it says so.

Stage	What it produces	Lands in
1 · Discover and probe	A grounded read of the problem, the users, and the data	Sprint 0
2 · Define	A PRD with scope and measurable success	Sprint 0
3 · Design	Architecture, agent topology, and a build-ready spec	Sprint 1
4 · Assess and mitigate risk	A risk register with concrete mitigations	Sprints 0 to 1, then maintained
5 · Data	Real datasets, a synthetic augmentation set, a benchmark	Sprints 1 to 2
6 · Build	The working agentic system, instrumented and observable	Sprints 2 to 3
7 · Verify	A full evaluation and QA pipeline	Every sprint, deepening in 3
8 · Operate	A deployed system with a runbook	Sprint 4

THE ENGAGEMENT

The client and the situation

Ironside Manufacturing runs a fleet of expensive, hard-working machines. When one fails without warning, the line stops, and an unplanned outage costs far more than a planned repair.

Each machine streams sensor data: temperatures, pressures, speeds, vibration, wear. Today, a small reliability team watches dashboards and reacts when something trips a threshold, which is often too late. The signal that a machine is heading for failure is usually there days earlier, buried in the data.

Ironside's leadership does not want another dashboard of red and green lights. They want a copilot that watches the whole fleet, catches degradation early, explains in plain language what is going wrong and how sure it is, and drafts the work order and the maintenance schedule for a human to approve. It has to catch the failures that matter, run across a fleet, and not drown the team in false alarms.

You are the AI engineering squad they have engaged to design, build, and defend it.

YOUR BRIEF

The mandate and the three lenses

> Take a real run-to-failure sensor dataset and build an agentic operations copilot. It monitors a fleet of assets, detects and explains degradation early, estimates remaining useful life, and drafts work orders and schedules for human approval. You carry the work through the full lifecycle, and you defend every decision at the end.

Three lenses run through everything you produce. The final presentation is organized around them, and the closing viva tests all three.

Product

The people you serve, the jobs they need done, the scope you commit to, and the metrics that prove the system is worth running.

Solution architecture

The agents and how they are arranged, the orchestration patterns you adopt and reject, and how the parts connect, recover, and stay within their roles.

Engineering

A system that holds under load, stays inside a cost budget, and carries the tracing, testing, and deployment a real operator insists on.

STAGE 1

Discover and probe

Before any architecture, understand the work. A system that does not match how the work actually arrives will fail in its first week. This stage is research, not building.

- Profile the real data. Quantify the distribution of assets, the most common categories, sizes, and any languages or formats present. Identify the small slice that drives most of the volume.
- Build three to five user personas: the people who depend on this system, the human expert who takes an escalation, and the operations lead who answers for cost and quality. Give each a goal and a frustration.
- Define the job to be done for each user in one sentence.
- Write the probing questions you would put to the client: volumes, peak patterns, what the system is allowed to do, and what must never be automated. Where you cannot ask, state the assumption and mark it.
- Tear down how two real industrial or monitoring products turn an alert into an action. Bring back one pattern worth borrowing and one anti-pattern.

> **DONE LOOKS LIKE**
> A short discovery brief, a persona set, and a prioritized map of the work, all backed by numbers from the data.

STAGE 2

Define

Turn the discovery into a contract. The PRD is the document a stakeholder could sign and an engineer could build against without a follow-up meeting.

- Write a PRD: problem statement, target users, jobs to be done, scope and non-scope, assumptions, and open questions.
- Define operational meanings for the key outcomes your system produces, so every later metric is unambiguous.
- Set one north-star metric, supporting metrics, and at least two guardrail metrics (a safety or cost ceiling that must never be crossed even when the headline metric improves).
- Put a number and a baseline against every metric. A target without a baseline cannot be judged.
- State the non-functional requirements: a latency budget, a per-unit cost ceiling, and the safety bar for sensitive cases.

Target metric	What it measures
Early-detection lead time	How far ahead of failure the system raises a credible alert.
False-alarm rate	How often it cries wolf; the team's trust depends on this.
RUL explanation quality	Whether the remaining-useful-life estimate and its reason are sound.
Work-order usefulness	Whether the drafted action is something a technician can act on.
Cost per asset monitored	The unit economics that decide whether this ships.

> **DONE LOOKS LIKE**
> A PRD a stakeholder could approve and a team could build from on its own.

STAGE 3

Design

Decide the shape of the system before writing it. Reviewers weigh why you arranged the agents this way as much as the diagram itself.

- Produce a system architecture diagram: the components, the path a asset takes through them, and where control passes between agents.
- Define the agent topology. Name each agent, its single responsibility, the inputs it accepts, the outputs it returns, the tools it may call, and the model tier it runs on.
- Write an orchestration decision record. Weigh a hierarchical arrangement (a fleet supervisor over per-asset agents over diagnostic and planning sub-agents), parallel processing (many assets watched concurrently), and plan-and-execute (the maintenance scheduler). State which you adopt, which you reject, and why. Scale is central: the topology must hold across a fleet.
- Specify agent contracts as schemas, so any agent can be swapped without breaking the system.
- Design the LLM gateway: the single point every model call passes through, with provider keys, rate-limit handling, timeouts, retries, and fallbacks.

- Design the model-routing policy as a decision table: which task goes to a cheap model, which to a stronger one, and the fallback chain when a model is slow or unavailable.
- Design the agent registry: how each agent is registered, versioned, discovered, and invoked, and the metadata it carries.
- Design memory and state: what is held within one interaction, and what persists across interactions.
- Specify the tools: an anomaly and remaining-useful-life model you build on the sensor data, a maintenance-manual retrieval tool, a work-order generator, and a scheduling tool. The agentic layer reads the models, explains them, and turns them into action.
- Capture all of it in a spec document.

> **DONE LOOKS LIKE**
> A spec a new engineer could build from on their own, without asking you questions.

STAGE 4

Assess and mitigate risk

An agent that acts on real money, real data, or a real machine can cause real harm. Treat risk as a first-class deliverable, not a closing paragraph.

- Build a risk register. For each risk, record likelihood, impact, an owner, a mitigation, and the residual risk that remains after the mitigation.

Risk	Why it matters	Direction for mitigation
Missed critical failure	A false negative on a critical fault is the worst outcome.	Bias toward recall on critical modes, conservative thresholds, escalation.
Alarm fatigue	Too many false alarms and the team stops trusting it.	Precision tuning, severity ranking, suppression of low-value alerts.
Wrong or unsafe maintenance action	A bad work order wastes time or causes harm.	Human approval, grounded procedures, action guards.
Hallucinated diagnosis	An invented cause misleads the team.	Grounding in the manuals and the model output, refusal when unsure.
Scale and cost blow-up	Watching a fleet can be expensive.	Cheap models for routine monitoring, escalate only on signal, hard budgets.

- Tie each top risk to a specific design decision that reduces it.
- Define a kill switch and a graceful fallback: when in doubt, the system degrades to a human path rather than guessing.

> **DONE LOOKS LIKE**
> A register where every high risk has a concrete mitigation mapped to the design, and a defined fallback.

STAGE 5

Data: real sources, synthetic augmentation, benchmarking

Start from real, public data, then make it production-realistic. Real data is narrow and clean; real work is broad and messy. You close that gap with synthetic data, and you prove the gap-closing worked with a benchmark.

Reference datasets (pick a primary, justify it)

Dataset	Source · scale · licence	Link
NASA C-MAPSS (Turbofan)	NASA PCoE via Kaggle · run-to-failure · 21 sensors · 4 condition regimes	kaggle.com/datasets/bishals098/nasa-turbofan-engine-degradation-simulation
NASA N-CMAPSS	NASA PCoE via Kaggle · realistic flight conditions · richer degradation	kaggle.com/datasets/bishals098/nasa-cmapss-2-engine-degradation
AI4I 2020 Predictive Maintenance	UCI · 10k records · 5 failure modes · CC BY 4.0	archive.ics.uci.edu/dataset/601/ai4i+2020+predictive+maintenance+dataset
Awesome Industrial Datasets	GitHub · a curated index of further public industrial and sensor datasets	github.com/jonathanwvd/awesome-industrial-datasets

Confirm each licence on its page. C-MAPSS is the benchmark run-to-failure dataset; N-CMAPSS adds realism; AI4I adds labeled failure modes for a classification angle.

Your demands at this stage

- Choose a primary dataset and justify the choice against your PRD.
- Produce a dataset card: source, licence, size, schema, class balance, known gaps, and sensitive-data considerations.
- Run exploratory analysis appropriate to the data.
- Build a synthetic-data pipeline. State the generation method, the prompt templates, and the controlled variation. Deduplicate, and validate with a mix of rules, an LLM check, and a human spot-check.
- Name what each synthetic set is for: rare fault signatures the real data barely contains, sensor drift and noise patterns, gradual versus sudden degradation profiles, and maintenance-log text the agents must read and write.
- Produce a synthetic-data card alongside the real one.
- Run a benchmark. Hold out a real test set. Measure quality on real-only versus real-plus-synthetic. Report the lift, and be honest where synthetic data made things worse.

> **DONE LOOKS LIKE**
> Two data cards and a benchmark table that shows, with numbers, what the augmentation changed.

STAGE 6

Build

Now make it run, and make it observable. A system you cannot trace or cost is not finished.

- Implement the agent topology end to end, with the orchestration patterns you chose in design.
- Bring the gateway live with working fallbacks, enforce the routing policy, and make the registry operational.
- Build the retrieval layer with a stated chunking and embedding choice, and wire the tools to mock services seeded from the data.
- Handle bursty load: run agents concurrently, queue work, and cache repeated retrievals and repeated model calls.
- Run synthetic-data generation and evaluation as distributed batch jobs that parallelize, not as one long serial loop.
- Instrument token economics. Record cost per call and per asset, surface a cost view, report cost per successful asset, and enforce a budget guardrail with alarms.
- Make the system observable. Trace every agent step and tool call, log every decision, and stand up a dashboard for throughput, cost, quality, and latency.
- Hold an engineering bar: tests, a CI check, clean environment management, and no secrets in code.

> **DONE LOOKS LIKE**
> A running system that survives a burst, traces a single asset end to end, and reports what it cost.

STAGE 7

Verify

Evaluation is not a final step. It runs every sprint and decides whether you are allowed to move on.

- Build a golden set from real labeled data plus curated correct outputs. Lock it and version it.
- Run it as a regression suite on every change, and let it gate merges.
- Use an LLM as a judge with a written rubric. Calibrate the judge against a small human-labeled sample and report how well the judge agrees with the humans.
- Use RAGAS to measure faithfulness, answer relevancy, and context precision and recall on the retrieval-grounded outputs.
- Use DSPy to optimize the prompts and pipelines against your metric, and report the before and after.
- Run A/B comparisons across routing policies, prompt variants, and agent configurations, judging quality and cost together, and pick winners with evidence.
- Run an end-to-end benchmark against a single-LLM baseline, and report the deltas honestly, including any regressions.

- Run a safety and red-team pass using your adversarial set, and report how well the system holds up.

> **DONE LOOKS LIKE**
> An evaluation report with a calibrated judge, RAGAS scores, A/B winners, and an honest benchmark against the baseline.

STAGE 8

Operate

Ship it, and write down how to run it. The handover is part of the work.

- Deploy the system as an API and a minimal interface, reproducibly.
- Write an operate runbook: monitoring and alerts, the human-fallback path, rollback, on-call, cost alarms, and an incident response.
- Define service-level objectives for latency, availability, and a cost ceiling, and the alerts that protect them.
- Define a canary and rollback plan for any prompt or model change, so a bad change can be reversed quickly.

> **DONE LOOKS LIKE**
> A deployed system and a runbook another team could operate without you in the room.

RAISING THE CEILING

Frontier techniques worth reaching for

None of this is required for a strong submission. It is how a strong submission becomes an exceptional one. Adopt only what genuinely serves your system; a protocol or a fine-tune bolted on for show will cost you in the viva.

Interoperability protocols (the emerging agent stack)

- **MCP** (Model Context Protocol): a standard way for an agent to reach tools and context, so your tools stay portable across clients.
- **A2A** (Agent2Agent, Google): a standard for agents to discover and call other agents across frameworks and vendors, using a JSON agent card over HTTP. Reach for it if your specialists could be independent services.
- **AG-UI** (Agent User Interaction Protocol): an event-based standard that connects an agent backend to a live frontend for real-time, streaming interaction.
- **A2UI** (agent-generated UI, built on A2A): lets an agent return interactive components, a chart, a form, an approve-or-reject control, instead of plain text. Use it where the right answer is an interface, not a paragraph.

Adaptation and fine-tuning (beyond prompting)

- **DoRA** (Weight-Decomposed Low-Rank Adaptation): an improvement on LoRA that separates a weight into magnitude and direction and adapts them apart, usually closing more of the gap to full fine-tuning at the same parameter budget. Prefer DoRA over plain LoRA when you fine-tune a small open model.

- **QLoRA**: quantized LoRA, so you can fine-tune larger models on modest hardware.
- **Preference tuning** (DPO, ORPO): align a model to preferred outputs without a full RLHF stack.
- **Distillation**: train a small, cheap model to imitate a strong one for the high-volume path.

Retrieval and grounding (beyond basic RAG)

- **Agentic RAG**: let an agent decide when and what to retrieve, and iterate.
- **Corrective RAG (CRAG)** and **Self-RAG**: judge the evidence, then re-retrieve or abstain when it is weak.
- **GraphRAG**: build a knowledge graph for multi-hop questions.
- **Reranking** (cross-encoders, ColBERT) and **HyDE**: sharpen what reaches the model.

Reasoning and orchestration (beyond a single chain)

- **ReAct** and **Reflexion**: interleave reasoning, action, and self-correction.
- **Plan-and-execute**: a planner sets the steps, an executor runs and replans.
- **Mixture-of-Agents** and **multi-agent debate**: combine or contest several agents on the harder calls.

Optimization and evaluation (beyond hand-tuned prompts)

- **DSPy**: compile and optimize prompts and pipelines against a metric. It is in your core stack; push it further here.
- **GEPA** and **TextGrad**: newer prompt and pipeline optimizers worth a head-to-head.
- **promptfoo**: a harness for systematic prompt and model evaluation.

Serving and scale (beyond a single process)

- **vLLM** and **speculative decoding** for fast, cheap inference of open models.
- **Quantization** (AWQ, GPTQ, GGUF) to shrink models for the cheap path.
- **Semantic caching** and KV-cache reuse to cut repeated cost.
- **Learned model routing** (RouteLLM-style) to send each request to the cheapest model that still passes.

Safety (beyond a single critic)

- **Guardrails** (Llama Guard, NeMo Guardrails) and structured-output constraints for safer, well-formed responses.

> **Where these land here.** A2A lets per-asset agents act as services in a fleet mesh; A2UI returns a maintenance work-order card and a schedule control; DoRA fine-tunes a small model on maintenance-log language; vLLM and quantization keep high-volume monitoring inference cheap; and plan-and-execute drives the scheduler.

CADENCE

How your weeks run

Five sprints, one week each. Each sprint opens with planning and closes with a review and a retro. A four-week run merges Sprint 0 and Sprint 1.

Sprint	Focus	Exit criteria (you may not move on until these are met)
0	Discover and Define	Profiled data and prioritized map, personas, PRD v1 with metrics and baselines, risk register v1, evaluation plan.
1	Design and De-risk	Architecture and topology, build-ready spec, gateway, routing and registry design, synthetic pipeline v1, evaluation harness and golden set, a working spike on the riskiest assumption.
2	Build the Core	The core agent path working end to end, gateway and routing live, first evaluation and cost baseline.
3	Harden, Scale, Optimize	Full agent set, concurrency and caching, A/B and DSPy results, regression suite, updated risk register.
4	Verify, Operate, Present	Benchmark against baseline, deployment and runbook, the 20-minute video, the viva, Demo Day.

HANDOVER

Final submission

- A structured code repository: README, architecture diagram, setup and run instructions, tests, and no secrets committed.
- The product and design pack: the final PRD, the spec, and the risk register with mitigations.
- The data pack: the real-dataset card, the synthetic-data card, and the augmentation and benchmark notes.
- The evaluation and QA report: RAGAS, the calibrated LLM judge, DSPy results, A/B outcomes, the safety pass, and an honest gap analysis against your PRD metrics.
- The engineering report: the architecture, the patterns chosen and rejected, the gateway and routing policy, the agent registry, the scale design, the token-economics report, and the observability setup.
- A deployed system, or a one-command reproducible deployment, with the operate runbook.

DEFENSE, PART ONE

The 20-minute presentation

One recorded presentation, organized around the three lenses. Live questions follow at Demo Day.

Segment	Time	Covers
Product	~6 min	The problem, the users, the discovery-to-PRD story, the metrics, and a live demo.
Solution architecture	~7 min	The topology, the patterns and the reasoning, the data flow, the gateway, routing and registry, and how risk shaped the design.
Engineering	~7 min	The build, the data and synthetic pipeline, distributed processing, token economics, the full evaluation pipeline with results against the baseline, deployment, and honest limitations.

DEFENSE, PART TWO

The one-on-one viva

The presentation defends the team's work. The viva confirms that you, individually, understand it.

- At the end of the capstone, each team member sits a fifteen-to-twenty-minute one-on-one viva.
- The examiner can open any part of the system. Expect to walk through code you did not personally write, justify a design tradeoff, explain a token-economics decision, and point to where the system would fail.
- Sample lines of questioning: why this orchestration pattern and not another; how the judge was calibrated and why you trust it; what a single successful asset costs and why; which risk worried you most and how you mitigated it.
- Prepare by being able to explain any decision in the build, not only your slice, and by rehearsing your reasoning out loud.

The viva exists to confirm genuine, even understanding across the team, and to surface work that was generated but not understood.

HOW YOU ARE JUDGED

Assessment

The team earns a score across five dimensions. The viva then sets an individual factor on top of it.

Dimension	Weight
Product and discovery	15%
Solution architecture	20%
Engineering and platform	25%
Quality and evaluation	25%
Demo Day and communication	15%

A team member who cannot defend the work in the viva does not inherit the team's score. The viva is an individual gate, not a formality.

TOOLING

Recommended open-source toolbox

Free-first. Use free models for all iteration, and reserve paid calls for evaluation and final runs.

Layer	Recommended (swap as you see fit)
Agent orchestration	LangGraph, CrewAI, or AutoGen
LLM gateway	LiteLLM
Models for development	Open free models, for example DeepSeek, Qwen, Llama, or a free Gemini Flash tier
Vector store	Chroma
Retrieval evaluation	RAGAS
Prompt and pipeline optimization	DSPy
Service and interface	FastAPI with a light front end
Deployment	A free cloud tier is sufficient for these systems